RUST

Introduzione al linguaggio

© G. Malnati, 2021-24

Introduzione

- Rust è un linguaggio di programmazione moderno focalizzato sulla **correttezza**, **velocità** e sul supporto alla **programmazione concorrente**
  - Tali obiettivi vengono perseguiti mantenendo ad un livello minimo le librerie di supporto in fase di esecuzione, in particolare senza usare tecniche di garbage collection né fare assunzioni sulla struttura dell'ambiente di esecuzione diverse da quelle del linguaggio C
  - Questo rende un programma scritto in Rust adatto ad essere eseguito in una varietà di contesti, dai sistemi embedded al kernel di un sistema operativo, dalle applicazioni lato server all'implementazione dei browser o di loro moduli
- Rust è un linguaggio **staticamente** e **fortemente tipato**, adatto alla programmazione di sistema
  - Ogni volta che introduciamo una espressione definiamo il tipo associato
  - Le variabili hanno un tipo che mantengono per tutta la loro durata in vita
  - Le funzioni hanno i parametri tipati e possono essere invocate solo assegnando valori coerenti con i tipi dei parametri
  - Tutti i tipi sono noti in fase di compilazione
  - Un sofisticato motore di **inferenza** viene usato per validare le proprietà dei tipi nel contesto del programma e ridurre significativamente il rischio di errori (in molti casi si può omettere l'annotaione di tipo laddove è deducibile dal contesto). Il tipo va specificato solo in caso di ambiguità
  - Compilatore molto preciso nell'individuazione dell'errore con suggerimento sulla correzione.
  - Permette un controllo totale dell'uso della memoria e ottimizza al massimo il codice generato

Storia

- Nato nel 2006 come progetto personale di Graydon Hoare (canadese), un dipendente di Mozilla Research, nel 2009 ha ricevuto una prima sponsorizzazione da parte di Mozilla, poi è diventato open source
  - Rust 1.0 - 15 maggio 2015
  - Ecosistema di sviluppo: modello di evoluzione basato sulla collaborazione di una comunità
  - **Versioni** successive rilasciate costantemente ogni 6 settimane
    - Ultima: 1.93.1 (12 febbraio 2026) [major.minor.patch]
    - Un anno fa …: Rust 1.85.0 - 17 febbraio 2025
  - Ogni 3 anni c'è una evoluzione nella sintassi con una nuova **edizione** (ver. 1.0 = RUST 2015, 2018, 2021, 2024).
- Usato in produzione in molti contesti
  - Firefox contiene 3M LoC scritte in Rust (9%)
  - Dropbox usa Rust nel suo motore di sincronizzazione
  - npm usa Rust per implementare il servizio di autorizzazione del registry
  - deno, il successore di node.js, è scritto in Rust
  - Amazon Web Services usa Rust (Firecracker project) per ottenere (alte prestazioni in servizi come Lambda, EC2, S3,...
  - Microsoft ha portato tutta l'API Windows in Rust (https://github.com/microsoft/windows-rs)
  - Google predilige Rust per lo sviluppo di codice nativo in Android

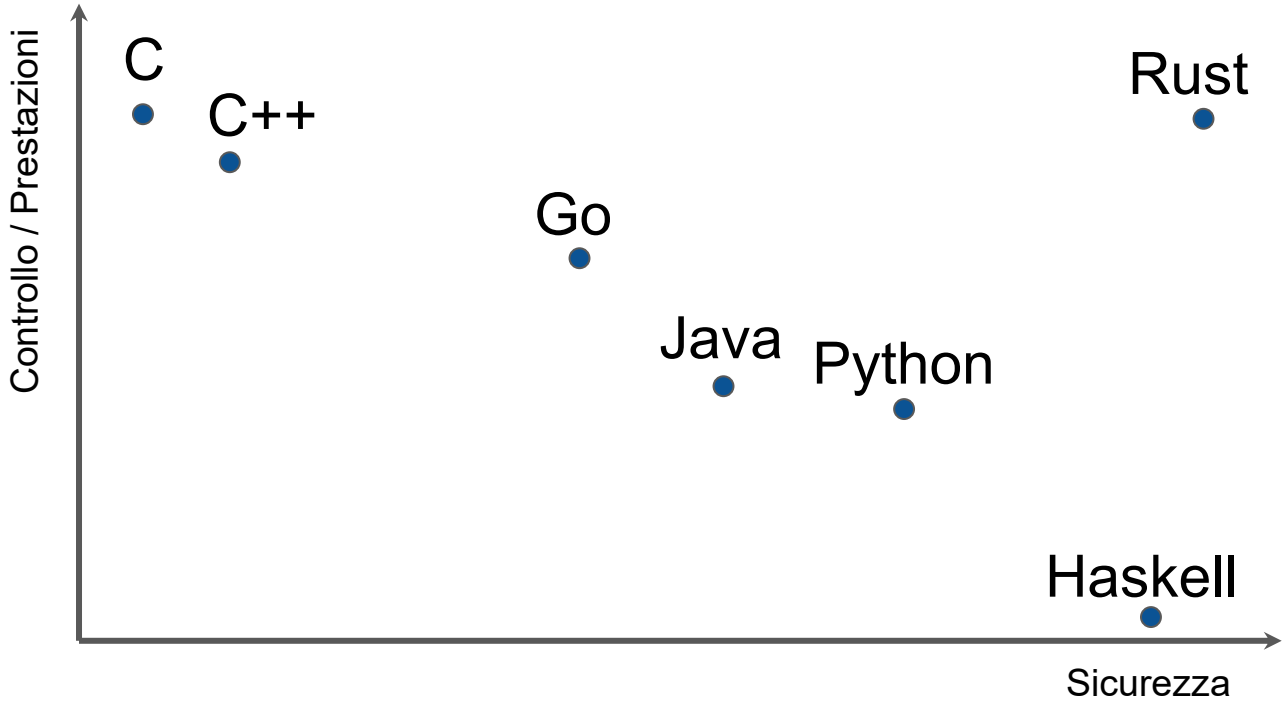
Obiettivi del linguaggio

- Offrire un linguaggio per la programmazione di sistema **privo di comportamenti non definiti, concorrente, pratico**
  - I linguaggi esistenti, ad un livello di astrazione ed efficienza simile, sono soggetti a molteplici limiti:
    - Poca attenzione alla sicurezza (safety) dei costrutti
    - Scarso supporto alla correttezza formale dell'esecuzione concorrente
    - Mancano di strumenti di corredo a supporto della creazione, condivisione e messa in campo
    - Offrono un controllo limitato sull'uso delle risorse computazionali
- Offrire **astrazioni a costo nullo** (*non devi pagare per quello che non usi*) per la maggior parte degli idiomi di programmazione
  - Permettendo al programmatore di adottare lo stile preferito nella scrittura di un algoritmo (iterazione, ricorsione, chiusure, …) e garantendo la generazione del miglior codice assembler possibile, senza introdurre costi aggiuntivi per funzionalità non richieste
- Supportare la **produttività del programmatore**
  - Offrendo costrutti di alto livello ed un ecosistema di compilazione/gestione delle dipendenze/test integrato (**cargo**)

Presupposti di base

- Linguaggio compilato (non basato su bytecode):
  - La compilazione genera un file eseguibile che contiene tutto ciò di cui ha bisogno e monta le librerie senza dipendenze con librerie esterne (DLL)
- Fortemente tipizzato in fase di compilazione
- Paradigma imperativo, ma con aspetti funzionali
- Programmazione concorrente
- Non ha né garbage collection né ambiente di supporto all'esecuzione
- Sistema dei tipi sofisticato

Confronto


Controllo / Prestazioni
C
C++
Go
Java
Python
Rust
Haskell
Sicurezza

Popolarità

Size of programming language communities in Q3 2023

Number of developers (in millions)

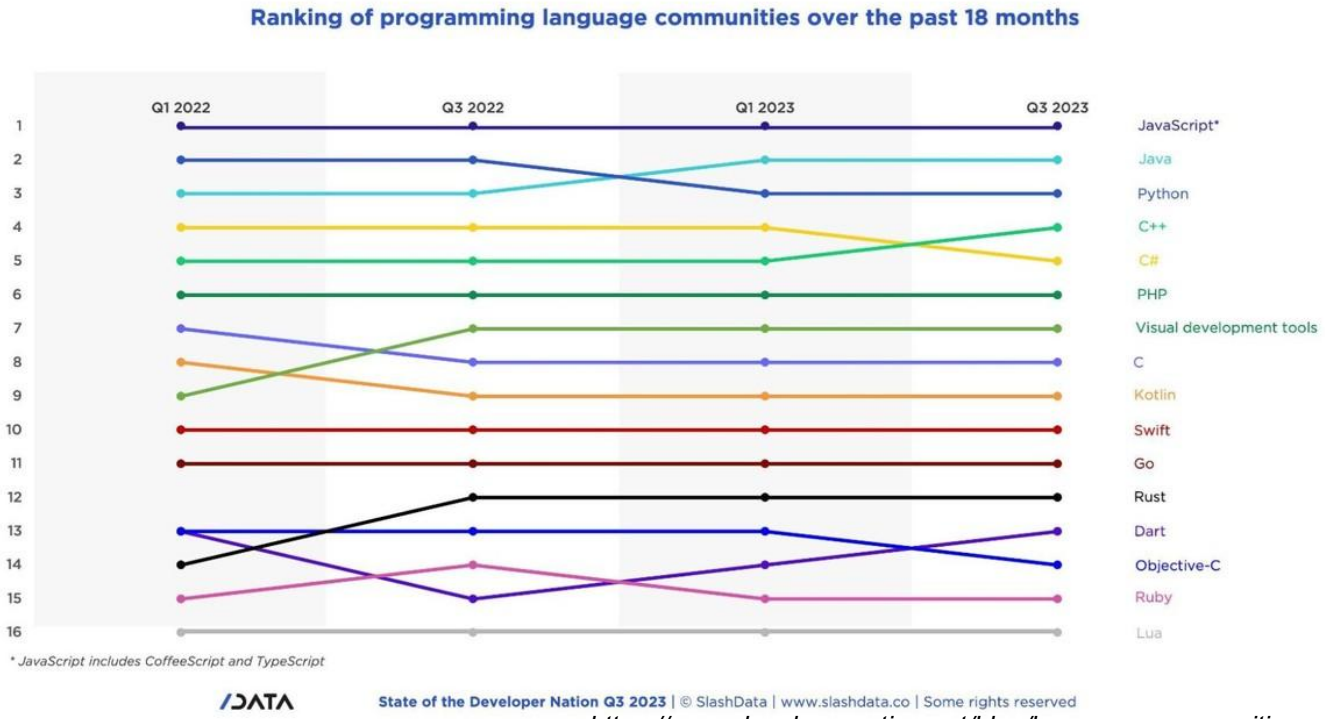
		Most popular in	Least popular in
JavaScript*	22.5M	Web, Cloud	Embedded, Device-level IoT coding
Java	17.5M	Cloud, Mobile	DS/ML/AI, Embedded
Python	16.9M	DS/ML/AI, app-level IoT coding	Games, Mobile
C++	10.3M	App-level IoT coding, Device-level IoT coding	Cloud, Web
C#	10.2M	Desktop, AV/AR	Mobile, DS/ML/AI
PHP	10.2M	Web, Cloud	Mobile, DS/ML/AI
Visual development tools	6.7M	Games, AR/VR	Device-level IoT coding, Cloud
C	5.9M	Device-level IoT coding, Embedded	Mobile, Web
Kotlin	5.1M	Mobile, AR/VR	Desktop, DS/ML/AI
Swift	4.5M	Mobile, AR/VR	App-level IoT coding, Cloud
Go	3.8M	Cloud, AR/VR	DS/ML/AI, Mobile
Rust	3.5M	App-level IoT coding, Embedded	DS/ML/AI, Mobile
Dart	2.9M	AR/VR, app-level IoT coding	Games, Desktop
Objective-C	2.5M	Mobile	Apps/extensions for 3rd-party platforms, Web
Ruby	2.3M	App-level IoT coding, Apps/extensions for 3rd-party platforms	Embedded, Web
Lua	1.6M	AR/VR, Device-level IoT coding	Desktop, Mobile

** JavaScript includes CoffeeScript and TypeScript*

/DATA **State of the Developer Nation Q3 2023** | © SlashData | www.slashdata.co | Some rights reserved

https://www.developernation.net/blog/language-communities-an-update

Popolarità



https://www.developernation.net/blog/language-communities-an-update

vs Python

- Molto più veloce
  - Normalmente destinato a contesti differenti
- Minor consumo di memoria
- Vero multi-threading
  - Parzialmente disponibile a partire da Python v.3.12
- Tipi algebrici
- Approccio all'ereditarietà differente
- Pattern matching
  - Introdotto in Python v.3.10
- Linguaggio staticamente orientato ai tipi: molti meno arresti anomali in fase di esecuzione
  - La gestione dinamica dei tipi usata da Python porta l'interprete ad accettare costrutti che si rivelano incoerenti solo in fase di esecuzione

vs Java

- Nessun overhead causato dalla JVM
  - non si verificano pause causate dal Garbage Collector
- Minor consumo di memoria
- Nessun costo di astrazione
- Approccio all'ereditarietà ed alla programmazione generica differente
- Pattern matching
- Sistema di compilazione unico
- Gestione delle dipendenze integrata

© G. Malnati, 2021-24

vs C/C++

- Nessun segmentation fault
- Nessun buffer overflow
- Nessun null pointer
- Nessuna corsa critica
- Sistema dei tipi più elaborato
- Approccio all'ereditarietà differente
- Processo di costruzione unificato
- Gestione delle dipendenze integrata

© G. Malnati, 2021-24

vs Go

- Nessuna pausa a causa del Garbage Collector
- Minor consumo di memoria
- Nessun null pointer
- Migliore gestione degli errori
- Programmazione concorrente sicura
- Sistema dei tipi più solido
- Approccio all'ereditarietà differente
- Nessun costo di astrazione
- Gestione delle dipendenze

© G. Malnati, 2021-24

Confronto (prestazioni *)

Total

	Energy		Time		Mb
(c) C	1.00	(c) C	1.00	(c) Pascal	1.00
(c) Rust	1.03	(c) Rust	1.04	(c) Go	1.05
(c) C++	1.34	(c) C++	1.56	(c) C	1.17
(c) Ada	1.70	(c) Ada	1.85	(c) Fortran	1.24
(v) Java	1.98	(v) Java	1.89	(c) C++	1.34
(c) Pascal	2.14	(c) Chapel	2.14	(c) Ada	1.47
(c) Chapel	2.18	(c) Go	2.83	(c) Rust	1.54
(v) Lisp	2.27	(c) Pascal	3.02	(v) Lisp	1.92
(c) Ocaml	2.40	(c) Ocaml	3.09	(c) Haskell	2.45
(c) Fortran	2.52	(v) C#	3.14	(i) PHP	2.57
(c) Swift	2.79	(v) Lisp	3.40	(c) Swift	2.71
(c) Haskell	3.10	(c) Haskell	3.55	(i) Python	2.80
(v) C#	3.14	(c) Swift	4.20	(c) Ocaml	2.82
(c) Go	3.23	(c) Fortran	4.20	(v) C#	2.85
(i) Dart	3.83	(v) F#	6.30	(i) Hack	3.34
(v) F#	4.13	(i) JavaScript	6.52	(v) Racket	3.52
(i) JavaScript	4.45	(i) Dart	6.67	(i) Ruby	3.97
(v) Racket	7.91	(v) Racket	11.27	(c) Chapel	4.00
(i) TypeScript	21.50	(i) Hack	26.99	(v) F#	4.25
(i) Hack	24.02	(i) PHP	27.64	(i) JavaScript	4.59
(i) PHP	29.30	(v) Erlang	36.71	(i) TypeScript	4.69
(v) Erlang	42.23	(i) Jruby	43.44	(v) Java	6.01
(i) Lua	45.98	(i) TypeScript	46.20	(i) Perl	6.62
(i) Jruby	46.54	(i) Ruby	59.34	(i) Lua	6.72
(i) Ruby	69.91	(i) Perl	65.79	(v) Erlang	7.20
(i) Python	75.88	(i) Python	71.90	(i) Dart	8.64
(i) Perl	79.58	(i) Lua	82.91	(i) Jruby	19.84

*** da:** Rui Pereira, et. all. 2017. **Energy efficiency across programming languages: how do energy, time, and memory relate?** In *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering* (*SLE 2017*). Association for Computing Machinery, New York, NY, USA, 256–267. DOI:https://doi.org/10.1145/3136014.3136031

Sicurezza

- Rust si appoggia ad un sistema di validazione dei tipi in fase di compilazione che impedisce (per i programmi che non fanno uso delle estensioni *unsafe*) i seguenti tipi di errore:
  - **Dangling pointer** - uso di puntatori ad aree di memoria GIÀ rilasciate
  - **Doppi rilasci** - tentativo di restituire al S.O. un'area di memoria già rilasciata
  - **Corse critiche** - accesso a dati il cui contenuto è indeterminato a seguito di eventi fuori dal controllo del programma stesso (ordine di schedulazione, attese legate all'I/O, …)
  - **Buffer overflow** - tentativi di accedere ad aree di memoria contigue a quelle possedute da una variabile, ma non di sua pertinenza
  - **Iteratori invalidi** - accesso iterativo (uno alla volta) agli elementi contenuti in una collezione che viene modificata mentre l'iterazione è in corso
  - **Overflow aritmetici** (solo in modalità debug) - esecuzione di operazioni aritmetiche che, a seguito della limitata capacità di rappresentazione dei numeri binari, portano ad errori grossolani (2_000_000_000 + 2_000_000_000 = -294_967_296)
- Il linguaggio favorisce l'uso di costrutti immutabili e propone convenzioni volte a limitare il rischio di compromissione dei dati

```rust
fn main() {
    let i = 2_000_000_000;

    println!("{}", i+i);
}
```

```rust
fn main() {
    let mut i = 2_000_000_000;
    let j = i;

    i = i + j;

    println!("{}", i);
}
```

overflow2.rs

overflow1.rs

```
thread 'main' panicked at src\main.rs:7:9:
attempt to add with overflow
```

```
--> src\main.rs:6:20
  |
6 |     println!("{}", i+i);
  |                    ^^^ attempt to compute `2000000000_i32 + 2000000000_i32`, which would overflow
  |
```

Prestazioni

- Il compilatore ottimizza in modo aggressivo **dimensioni** e **velocità** del codice generato
  - Le strutture dati base del linguaggio sono pensate per favorire l'**uso della memoria cache**, preferendo l'uso di array a più articolate strutture basate su puntatori che limitano la località dei riferimenti, vanificando i benefici statistici offerti dalla cache
  - La politica di invocazione standard è basata su **indirizzi statici**, che possono essere ottimizzati inline, piuttosto che su meccanismi polimorfici che richiedono l'uso di invocazioni indirette, non ottimizzabili
  - La presenza di un sistema integrato per la **gestione delle dipendenze** e dei moduli facilita la condivisione del codice e l'accesso ad un vasto panorama di librerie open source che spesso offrono soluzioni allo stato dell'arte per molti problemi comuni

© G. Malnati, 2021-24

Sicuro per definizione

- Il linguaggio introduce in modo esplicito il concetto di **possesso di un valore**
  - In ogni istante, ciascun valore è posseduto da una e una sola variabile
  - E' possibile trasferire il possesso di un valore ad un'altra variabile (movimento)
  - E' possibile concedere temporaneamente l'accesso ad un valore tramite l'uso di riferimenti, a patto di non modificare tale valore mentre il riferimento esiste
- Puntatori **controllati** in fase di compilazione
  - Il concetto di proprietà del dato è esteso ai dati cui si accede in modo indiretto, sia per riferimento (senza trasferimento della proprietà) che tramite puntatore
- La **sicurezza dei thread** è incorporata nel sistema dei tipi
  - Il compilatore conosce quali tipi possono essere trasferiti da un thread ad un altro e quali puntatori possono essere usati in modo sicuro tra thread
  - Il concetto di "tempo di vita" associato a ciascun valore permette di individuare in fase di compilazione quali operazioni sono lecite e quali portano ad una violazione
- Nessun **stato nascosto**
  - La presenza di errori e l'opzionalità sono modellate esplicitamente e richiedono il controllo esplicito da parte del programmatore

Controllo a basso livello

- La memoria è rilasciata non appena una variabile esce di visibilità
  - Nessuna interruzione a causa del garbage collection
- Nessuno spreco di memoria
- E' possibile invocare system call (tra cui fork/exec)
- Può essere eseguito su dispositivi senza sistema operativo
- Chiamate FFI verso altri linguaggi (C ABI)

© G. Malnati, 2021-24

Portale della Didattica - Pagina | Rust Programming Language

rust-lang.org

Laurea Magistrale | PEC | Gmail | IMMTORR | Rust Playground | RUST - code | Easy Rust | Tutti i preferiti

Install | Learn | Playground | Tools | Governance | Community | Blog | English (en-US)

Rust

A language empowering everyone
to build reliable and efficient software.

GET STARTED

Version 1.93.1

Why Rust?

Performance

Rust is blazingly fast and memory-efficient: with no runtime or garbage collector, it can power performance-

Reliability

Rust's rich type system and ownership model guarantee memory-safety and thread-safety — enabling you to eliminate

Productivity

Rust has great documentation, a friendly compiler with useful error messages, and top-notch tooling — an integrated

Installare Rust

- Si accede al sito https://rustup.rs e si seguono le istruzioni riportate
  - Per i sistemi operativi di derivazione Unix (Linux, MacOS, …) questo equivale a scaricare ed eseguire uno script che provvede a scaricare la toolchain e a configurare l'ambiente di esecuzione
  - Nel caso di Windows, vengono forniti due eseguibili, entrambi chiamati rustup-init.exe, destinati ad installare la toolchain nelle versioni a 64 o 32 bit

rustup.rs - The Rust toolchain in

rustup.rs

Laurea | Laurea Magistrale | WinterSession-II - D... | Informatica - Dropb... | Workshop Informati... | PEC | TIL | Gmail | University Overall R... | Tutti i preferiti

rustup

An installer for the systems programming language Rust

To install Rust, download and run
rustup-init.exe
then follow the onscreen instructions.

You may also need the Visual Studio prerequisites.

If you're a Windows Subsystem for Linux user run the following in your terminal, then follow the onscreen instructions to install Rust.

```
$ curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

You appear to be running Windows 64-bit. If not, display all supported installers.

Installare Rust

- Poiché il linguaggio (così come la libreria standard) viene aggiornato ogni 6 settimane, si allinea la configurazione del proprio PC con il comando
  - rustup update
- Sono disponibili diverse toolchain alternative
  - Nel caso di Windows, è possibile scegliere tra quella basata sul compilatore Microsoft (msvc) e quella basata su GCC/GDB
- L'installazione di una toolchain mette a disposizione vari strumenti, tra cui
  - **rustc** - il compilatore del linguaggio
  - **cargo** - lo strumento di gestione dei progetti e delle loro dipendenze

Cargo

- Programma ufficiale per la gestione di un package
- Documentazione: https://doc.rust-lang.org/cargo/guide/index.html
- Offre supporto per:
  - Indicazione dei metadati di progetto (nome, versione, autore, tipo di progetto, versione del linguaggio, …)
  - Gestione delle dipendenze (uso di altre librerie) e integrazione con il sito crates.io (registro ufficiale dei package Rust)
  - Gestione dei test di unità e di integrazione

- Si crea un nuovo progetto con i comandi:
  - **`cargo new`** ***`project_name`*** crea un progetto pensato per essere un eseguibile (con 1 punto di ingresso)
  - **`cargo new --lib`** ***`library_name`*** crea un progetto pensato per essere una libreria (con n punti di ingresso che sono le funzioni che qualcun altro potrà eseguire)
- Si compila / esegue un progetto con i comandi:
  - **`cargo build`**
  - **`cargo run`**

- Facendo **cargo new** viene generato:
  - Cartella **src** con un sorgente campione (**main.rs**)
  - File **cargo.toml**, file di configurazione principale per i progetti Rust e contiene una serie di sezioni (scritte dentro le parentesi quadre)
    - [package] descrive il progetto (nome, versione, autori, edizione di Rust)
    - [dependencies] dentro cui possiamo elencare le librerie di cui abbiamo bisogno
- Facendo **cargo build** viene generato:
  - File **cargo.lock** che dettaglia tutte le librerie che sono state installate
  - Cartella **target** dove viene messo l'eseguibile nella sottocartella **\target\debug**
- Facendo **cargo build –release** viene generato l'eseguibile nella sottocartella **\target\release**
- Facendo **cargo new -- lib** nella cartella **src** viene generato il file **lib.rs.**

Uso di librerie

- L'ecosistema di Rust è fortemente integrato
  - Chi crea una libreria può pubblicarla sul sito https://crates.io, rendendola disponibile a tutta la comunità degli sviluppatori
- Installando l'estensione cargo-edit (comando: **cargo install cargo-edit** https://github.com/killercup/cargo-edit), si può importare una libreria pubblica con il comando
  - `cargo add <libname>`
  - Esso aggiunge, nel file Cargo.toml, una sezione chiamata [dependencies] nella quale sono indicate, una per riga, le librerie importate e la relativa versione (quella più recente)
  - Questo fa sì che, all'atto della compilazione del package, le librerie da cui si dipende vengano automaticamente collegate al programma generato, rendendo così disponibili i relativi contenuti (tipi e funzioni)

IDE Web

- E’ possibile avere un assaggio della programmazione in Rust **senza dover installare nulla** tramite il sito https://play.rust-lang.org/
  - Offre un IDE web con la possibilità di accedere ad oltre 250 crates aggiuntivi mediante il quale è possibile scrivere, compilare ed eseguire codice Rust con alcune limitazioni (nessuna connessione di rete, memoria e disco limitati, vincoli sul tempo massimo di compilazione ed esecuzione)
  - I programmi scritti tramite il playground possono essere esportati sotto forma di GitHub GIST

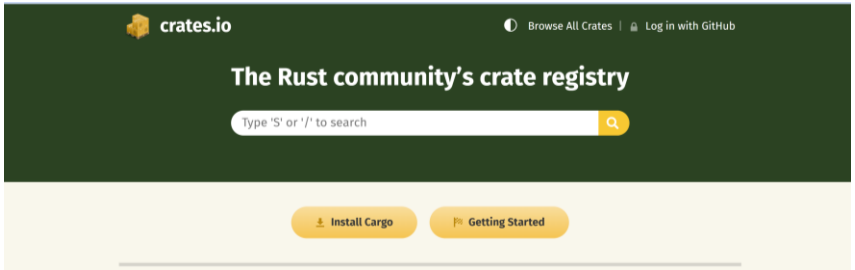
Ambienti integrati di sviluppo (IDE)

- Molti ambienti di sviluppo offrono supporto per lo sviluppo in Rust
- Tra gli altri
  - **JetBrains / RustRover - https://www.jetbrains.com/rust/nextversion/**
  - **VS Code - https://marketplace.visualstudio.com/items?itemName=rust-lang.rust**
  - Eclipse - https://www.eclipse.org/downloads/packages/release/2019-09/r/eclipse-ide-rust-developers-includes-incubating-components
  - Sublime Text - https://github.com/rust-lang/rust-enhanced
  - Atom - https://github.com/rust-lang/atom-ide-rust
  - NeoVim - https://dhghomon.github.io/easy_rust/

Terminologia

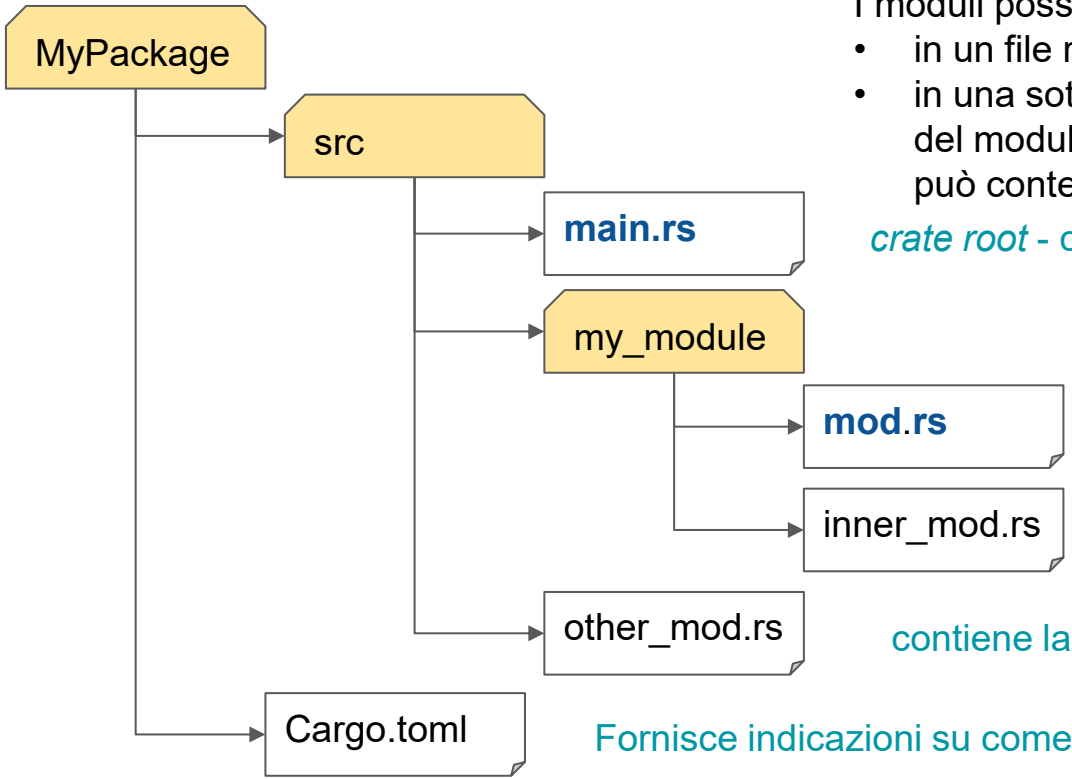
- **Crate**
  - **Unità di compilazione** che può dare origine ad un programma eseguibile (binario) o ad una libreria
  - Può contenere riferimenti a moduli contenuti in ulteriori file sorgenti: questi sono inclusi nel file corrente prima di avviarne la compilazione
- **Crate root**
  - File sorgente da cui parte il compilatore Rust per creare il modulo principale del crate
  - Un crate binario deriva da src/main.rs, una libreria deriva da src/lib.rs
- **Module**
  - Partizione del codice sorgente in blocchi (spazio dei nomi)
  - Meccanismo usato per suddividere gerarchicamente il codice sorgente in unità logiche differenti e regolarne la visibilità reciproca
  - Un modulo può contenere funzioni, tipi ed altri moduli
  - Un sotto-modulo può essere contenuto nel file sorgente del suo genitore, essere scritto in un file sorgente a parte o essere memorizzato in una sottocartella della cartella in cui è ospitato il suo contenitore
  - I moduli non sono compilati individualmente, ma solo come parte di un crate che li contiene
- **Package**
  - Insieme di uno o più crates volti a fornire un insieme di funzionalità (**progetto**)
  - Un package è ospitato in una cartella che contiene il file Cargo.toml: esso descrive come costruire i crate di cui è composto il package
  - Un package può contenere al massimo una libreria
  - Se contiene più crate binari, questi vengono posti nella cartella src/bin

Crates.io



- Si trovano tutte le librerie pubbliche disponibili per RUST
- Si possono caricare le proprie librerie (da account github)

Struttura di un progetto eseguibile Rust



I moduli possono vivere:

- in un file rs a fianco di main.rs, oppure
- in una sottocartella (nome del modulo) e il contenuto del modulo si deve chiamare mod.rs. La sottocartella può contenere dei sottomoduli del modulo

crate root - contiene il modulo principale

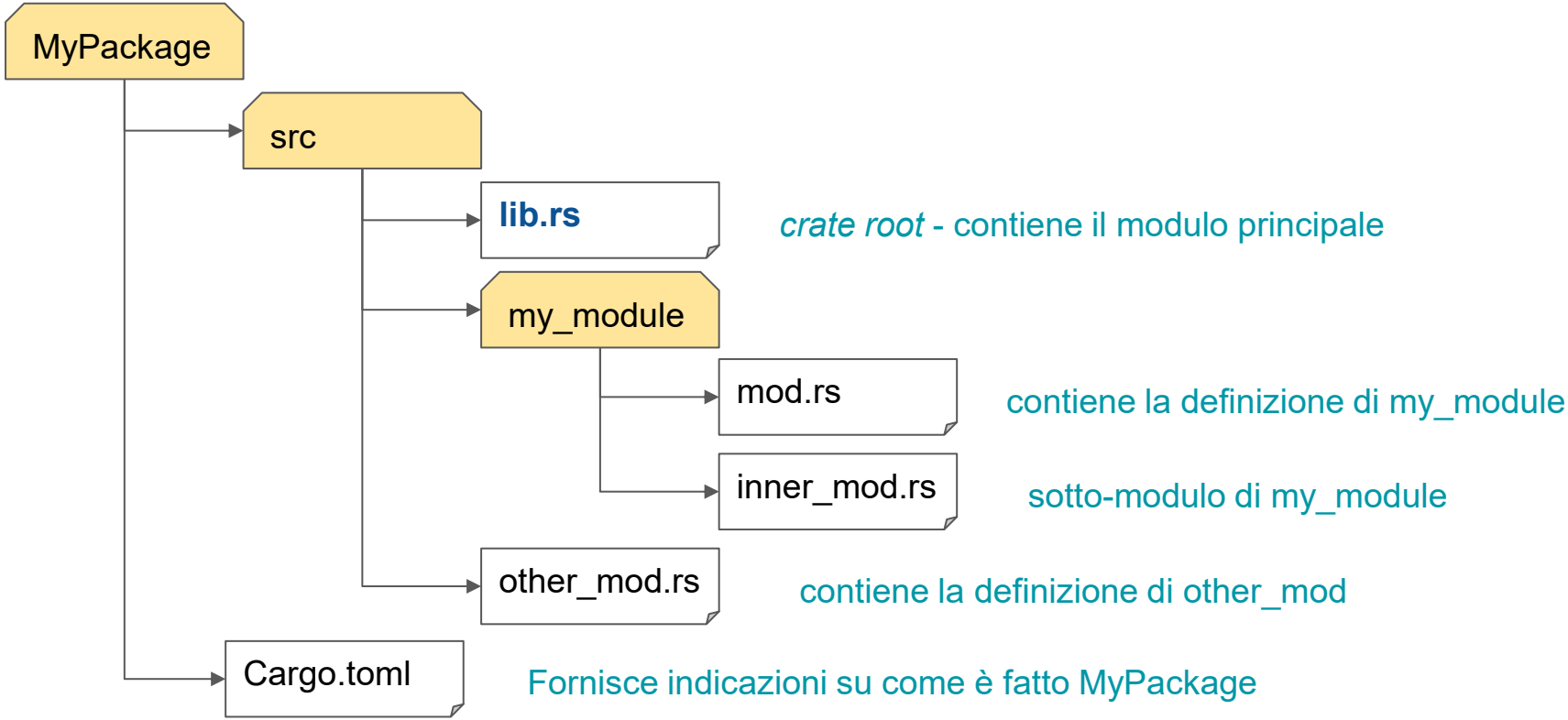
contiene la definizione di my_module

sotto-modulo di my_module

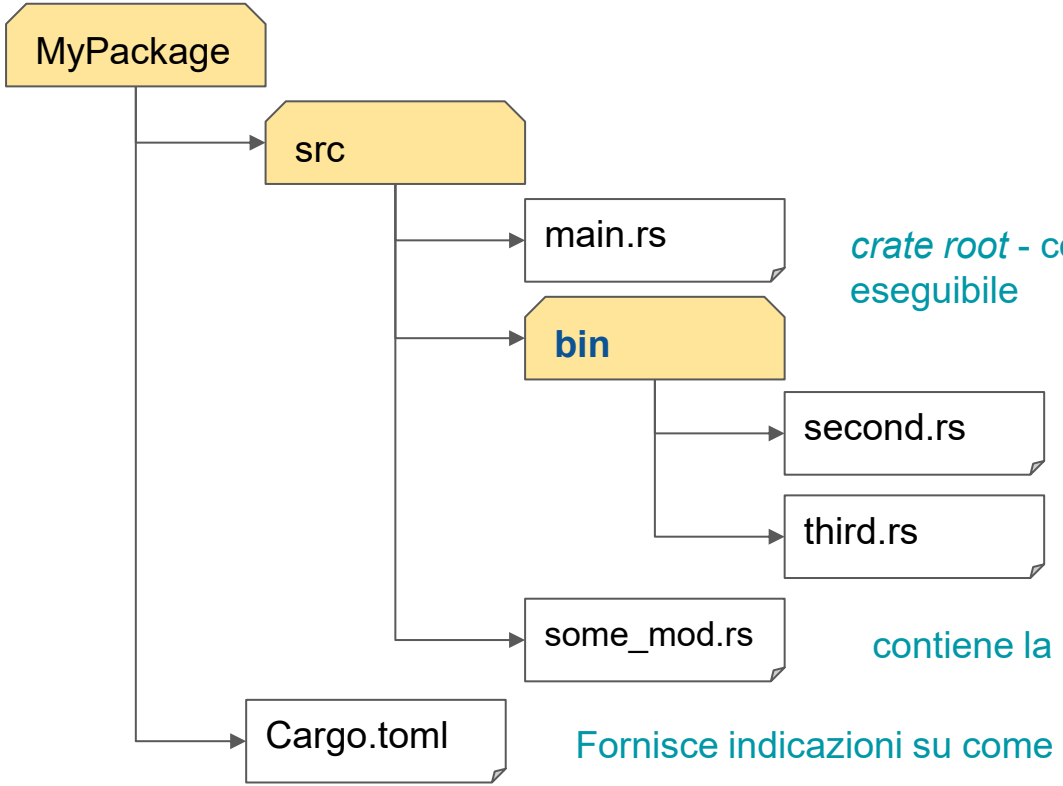
contiene la definizione di other_mod

Fornisce indicazioni su come è fatto MyPackage

Struttura di un progetto di libreria Rust



Struttura di un progetto con due o più eseguibili



crate root - contiene il modulo principale del primo eseguibile

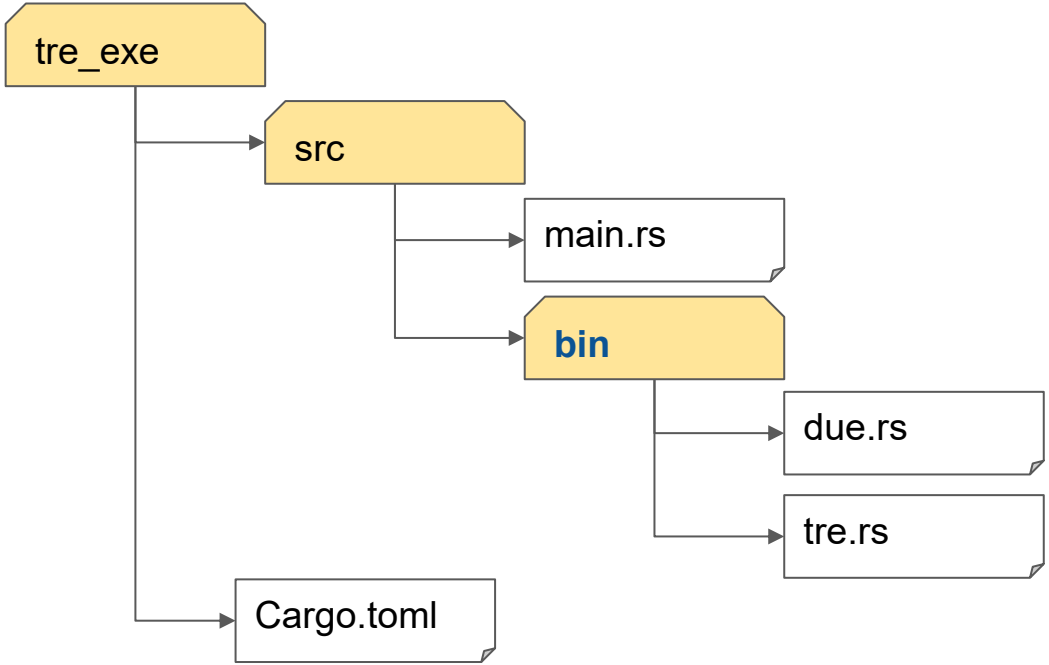
contiene il modulo principale del secondo crate eseguibile

contiene il modulo principale del terzo crate eseguibile

contiene la definizione di some_mod

Fornisce indicazioni su come è fatto MyPackage

Struttura di un progetto con due o più eseguibili: esempio



Cargo.toml

```
[package]
name = "tre_exe"
version = "0.1.0"
edition = "2024"

[[bin]]
name = "main"
path = "src/main.rs"

[[bin]]
name = "due"
path = "src/bin/due.rs"

[[bin]]
name = "tre"
path ="src/bin/tre.rs"
```

```
$ cargo run --bin tre
Compiling tre_exe v0.1.0 (/home/reba/tre_exe)
 Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.40s
    Running `target/debug/tre`
Salve Mondo!
$ cargo run --bin due
   Compiling tre_exe v0.1.0 (/home/reba/tre_exe)
   Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.41s
    Running `/target/debug/due`
Bonjour le monde!
$ cargo run --bin main
   Compiling tre_exe v0.1.0 (/home/reba/tre_exe)
   Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.50s
   Running `/target/debug/main`
Hello, world!
```

Sopravvivere in Rust

- Il punto di ingresso di un programma è `fn main() { … }`
  - **fn** funzione
  - Normalmente si trova nel file main.rs
- Per stampare su sul flusso di output *stdout*: `print!(...)` oppure `println!(...)`
  - Il primo parametro è una stringa di formato: per ciascun *placeholder* **{}** (positional argument) presente al suo interno deve essere indicato un parametro successivo, il cui contenuto sarà inserito al posto delle graffe
  - Tutto quello che non sta nelle graffe viene stampato così com'è
    `println!("Hello, {}!" , "world"); // → Hello, world!`
  - Per stampare sul flusso *stderr*: `eprint!(...)` oppure `eprintln!(...)`
- Si dichiarano le variabili con la parola chiave let
  - Nella maggior parte dei casi, il compilatore è in grado di dedurre automaticamente il tipo associato
  - `let x : i32 = 13;`
  - `println!("{}", x);`

La famiglia degli interi

- Quanto è un grande un intero (`int`) in C?

- In Rust le variabili hanno una grandezza ben definita:
  - **`i32`**: intero con segno su 32 bit
  - **`u32`**: intero senza segno su 32 bit
  - e analogamente **`i8, i16, i64, i128, u8, u16, u64, u128`**

- Rust fornisce anche i tipi **`isize`** e **`usize`** per interi con o senza segno grandi quanto il parallelismo dell'architettura corrente.

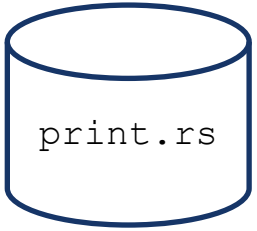
Let e let mut

- Let dichiara una variabile con possesso immutabile e non permette un cambiamento
- Let mut dichiara una variabile che può cambiare valore.

- Vale la regola del minor privilegio possibile (di default sono immutabili).

println!(): formattazione

```
fn main() {
    let num = -1;
    let ptr = &num;
    println!("x: {}", num);
    println!("ptr: {:x}", ptr);
    println!("indirizzo: {:p}", ptr);
}
```



- `println!("{:x}", num);`
  - Stampa del valore in formato esadecimale
- `println!("{:p}", ptr);`
  - Stampa del valore come puntatore.

Sopravvivere in RUST (I)

```
// Read a line from the console
Use stdio::io;
fn main() {
 println!("Type a name");
 let mut name = String::new();
 io::stdin()
  .read_line(&mut name)
  .expect ("Error reading line");
 println!("Hello {}!",name);
}
```

console.rs

- Per leggere dalla console devo accedere al flusso di input che vive nel *namespace* std::io (fa parte della libreria standard std e nell'ambito della libreria riguarda la parte di I/O)
- use stdio:io; Permette di importare le funzioni ed i tipi che vivono in quel namespace.
- let mut name = String::new();
  - Permette di allocare uno spazio di memoria per caricare la stringa letta da console, attraverso la funzione creatrice di una stringa, che ritorna una stringa vuota
  - variabile mutabile perché inizialmente vuota, ma da riempire
- io::stdio() mi fa accedere al flusso di ingresso
- .read_line() è una funzione che permette di leggere da un flusso tanti caratteri quanti sono necessari fino arrivare al fine riga e metterli nella variabile name, passata per puntatore alla funzione, dando il permesso di scriverci dentro (&mut). Viene fornito il puntatore da usare per scrivere
- L'operazione di I/O può fallire: read_line() restituisce un Result che dice se l'operazione è andata bene oppure è fallita, se non mettessi nulla a valle della read_line il compilatore segnala errore per la mancanza di gestione del Result (operazione che potrebbe fallire non testata)
- .expect("Error reading line"); se si è verificato un errore, termina il programma, stampando il messaggio inserito come parametro
- Se va tutto bene, quello che viene letto da console viene menorizzato in name

Sopravvivere in RUST (II)

```
// Read a file line by line
use std::fs::File;
use std::io::BufReader;
use std::io::BufRead;

fn main() {
    let file = File::open("hello.txt")
        .expect("File error")
        ;
    let reader = BufReader::new(file);

    for line in reader.lines() {
        println!("{}", line.
          expect("Error reading file"))
    }
}
```

file.rs

- Leggere da File, che è un blocco di byte referenziato con un nome e memorizzato su disco. Il nome del file è caratterizzato da un percorso (pathname)
- Use std::fs::File permette di importare le funzioni utili per accedere ai file
- let file = File::open("hello.txt"); serve per aprire un file con il nome specificato, cercandolo nella cartella corrente
- .expect() se open non trova il file, il programma viene abortito con un messaggio di errore
- file rappresenta un puntatore all'oggetto file. Per leggerlo devo costruire un'altra rappresentazione, basata su un buffer per la lettura (BufReader)
- let reader = BufReader::new(file); crea una variabile di tipo BufReader per fare la lettura del file a blocchi
- metodo lines() è un iteratore che mi permette di scandire il file a partire dall'inizio e andando a leggere una riga alla volta fino a raggiungere la fine del file e mentre lo legge lo mette nella variabile line, se l'operazione ha avuto successo, altrimenti abortisce il programma con un messaggio di errore

Sopravvivere in RUST (III)

write.rs

```
// Write a file

use std::fs::File;
use std::io::Write;
fn main() {
    let mut file = File::
       create("output.txt")
       .expect(" File error")
        ;

    file
      .write_all("Hello!".as_bytes())
      .expect("unable to write");
}
```

- Apre un file in scrittura e dunque devo creare una variabile *mutabile* utile per accedere in scrittura ad un file
- Si utilizza il metodo create per creare un file nel pathname specificato (se il file esistesse, viene cancellato)
- Se qualcosa non funziona, expect() abortisce l'esecuzione con un messaggio di errore
- Grazie alla variabile che fa riferimento al file va a scrivere (`write_all`) la stringa passata come parametro, avendola convertita in sequenze di byte (`write_all` è pensata per scrivere tutta la stringa passata come parametro e non solo una parte).

Sopravvivere in RUST (IV)

```
// List directory contents
use std::fs;

fn main() {
    let entries = fs::read_dir("./")
       .expect("Failed to read dir");

    for entry in entries {
        let path =
          entry
            .expect("Failed to read")
            .path();

    println!("{}", path.display());
    }
}
```

list.rs

- Dal modulo `fs` prende la funzione `read_dir()` e leggi il contenuto dalla cartella specificata dal parametro (in questo caso è la cartella corrente) e lo mette nella variabile `entries`.
- Il ciclo for scandisce tutte i dati che ha acquisito in entries e ne legge uno alla volta, verificando che sia corretto e fornendo il path (`path()`).
- Infine stampa il path con la funzione `display()` che converte il path in stringa.

Per saperne di più

- Tour of Rust - https://tourofrust.com/
  - Tutorial interattivo disponibile molte lingue (italiano compreso) in grado di fornire i concetti essenziali della programmazione in Rust
- Rust in Easy English - https://dhghomon.github.io/easy_rust/
  - Altro tutorial interattivo, con stile più discorsivo e spiegazioni passo passo di tutti i principali concetti del linguaggio
- Rust Language Cheat Sheet - https://cheats.rs/#the-abstract-machine
  - Riassunto sintetico del linguaggio, corredato di esempi interattivi e riferimenti ad una grande quantità di risorse per gli approfondimenti
- Rust By Example - https://doc.rust-lang.org/stable/rust-by-example/
  - Raccolta ufficiale di esempi funzionanti che illustrano i diversi concetti del linguaggio e delle librerie standard
- Rust Cookbook - https://rust-lang-nursery.github.io/rust-cookbook/
  - Raccolta di esempi che presentano le buone pratiche del linguaggio applicato ad una ventina di contesti applicativi - richiede una conoscenza ragionevole del linguaggio

Per saperne di più

- The Book - https://doc.rust-lang.org/book/
  - Il tutorial ufficiale del linguaggio
- The Rust Reference - https://doc.rust-lang.org/stable/reference/
  - La guida di riferimento ufficiale del linguaggio: utile per approfondire specifici concetti dopo che si è imparata la struttura generale
- Rust-learning - https://github.com/ctjhoa/rust-learning
  - Una ampia collezione curata ed aggiornata di link ad articoli e video legati all'apprendimento di Rust, suddivisi per argomento
- The Little Book of Rust Books - https://lborb.github.io/book/
  - Collezione curata ed aggiornata di libri online su Rust e sul suo ecosistema
- Rust Books - https://github.com/sger/RustBooks
  - Raccolta di riferimenti a libri cartacei (e digitali) con link ai relativi editori e/o venditori

Risorse Utili (progetti e lavori in Rust)

- Amazon Firecracker Micro-VM
  - https://github.com/firecracker-microvm/firecracker
- Redox Operating System
  - https://gitlab.redox-os.org/redox-os/redox/
- TockOS (Embedded Operating System)
  - https://www.tockos.org
- Writing a OS in Rust (blog)
  - https://os.phil-opp.com/
- Bevy (simple data-driven game engine)
  - https://github.com/bevyengine/bevy
- SWC (Speedy Web Compiler)
  - https://github.com/swc-project/swc
- Jobs
  - https://rustjobs.dev/

Glossario

- **Type inference**: caratteristica fondamentale di Rust che permette al compilatore di dedurre automaticamente i tipi delle variabili, delle espressioni e delle funzioni quando non sono esplicitamente dichiarati
- **Versione ed edizione del linguaggio**: determinano l'evoluzione del linguaggio: le versioni sono rilasciate ogni 6 settimane, le edizioni sono rilasciate ogni 3 anni.
- **Cargo**: strumento ufficiale di gestione pacchetti e build per Rust
- **Paradigma imperativo**: stile di programmazione che si basa su istruzioni che cambiano lo stato del programma attraverso assegnazioni di variabili e caratterizzato dall'uso di strutture di controllo come cicli e condizioni per guidare l'esecuzione
- **Toolchain**: insieme degli strumenti necessari per sviluppare e compilare codice

© G. Malnati, 2021-24

Glossario (II)

- **Crate**: una unità di compilazione e distribuzione di codice. Può essere considerato come un pacchetto o una libreria che contiene uno o più moduli di codice sorgente. I crate sono la base del sistema di gestione dei pacchetti in Rust e sono utilizzati per organizzare e condividere il codice tra progetti
- **Crate root**: punto di partenza della compilazione ed è il file principale che definisce la base del codice da cui tutto il resto del crate viene costruito
- **Module**: struttura che permette di organizzare e separare il codice in unità logiche
- **Package**: raccolta di uno o più crate che vengono distribuiti insieme come singola unità di progetto

© G. Malnati, 2021-24

Glossario (III)

- **macro**: meccanismo che permette di generare codice in modo dinamico durante la compilazione.

© G. Malnati, 2021-24